# Lattice surgery in Stim: measuring $Z_A Z_B$ between two surface-code patches

Qiyu Zhang · Erdős Institute Quantum Computing Bootcamp, Mini-project 2

https://github.com/DerivedUsername/lattice-surgery-stim

## 1 Problem

The rotated surface code stores one logical qubit in a $d \times d$ grid of data qubits; checks of weight 2 or 4 are measured on the faces, and the code space is a joint eigenspace of the checks. Given two patches $A$ and $B$, we want to measure $Z_A Z_B$ without measuring $Z_A$ or $Z_B$. This reveals only the parity, so a state such as $\alpha|00\rangle + \beta|11\rangle$ keeps its superposition; it is the basic two-qubit operation between patches. Lattice surgery (Horsman et al., arXiv:1111.4022, §3 and §7.1) does it by merging the patches for $d$ rounds and splitting them again. We build this as a Stim circuit with circuit-level noise, decode it with PyMatching, and compare with two idle patches.

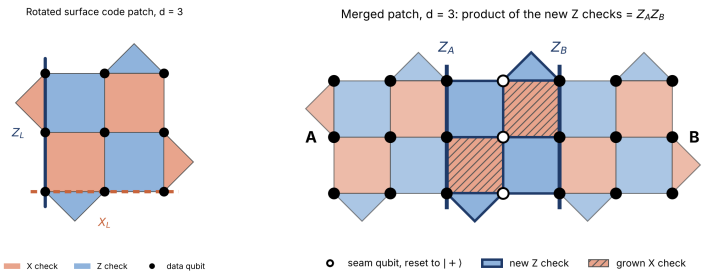



Figure 1: Left: a $3 \times 3$ patch. Right: the merged patch, with $d+1$ new $Z$ checks and $d-1$ grown $X$ checks.

## 2 Why the merge measures $Z_A Z_B$

A column of $d$ seam qubits between the patches is reset to $|+\rangle$, and the checks of the single $(2d+1) \times d$ patch $M$ are measured. Compared with $A$ and $B$, no check is removed, $d-1$ $X$ checks grow by $X_s X_{s'}$ on two seam qubits, and $d+1$ $Z$ checks are new. An unchanged check returns its old value, and so does a grown one, since $X_s X_{s'} = 1$ on the seam. A new check anticommutes with $X_s$ for a seam qubit $s$ in it, so on every code state its outcome is $\pm 1$ with probability $\frac{1}{2}$ and says nothing about $A$ and $B$. The product of the new checks is different: each seam qubit lies in exactly two of them, so its $Z$ cancels and

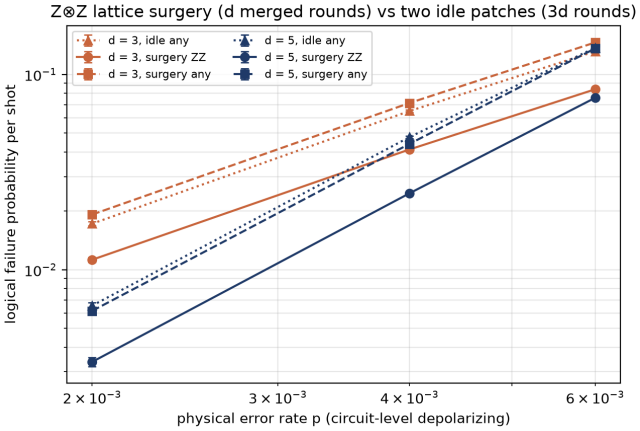
$$\prod_j n_j = Z_{\text{column } d-1} \, Z_{\text{column } d+1} = Z_A Z_B.$$

So the product $\lambda$ of the $d+1$ random outcomes is the value of $Z_A Z_B$. On the code space the merge acts as $\psi \mapsto U_\lambda P_\lambda \psi$: a projection onto the eigenspace $Z_A Z_B = \lambda$, then an isomorphism onto the code space of $M$, fixed up to a Pauli correction tracked in software. The operators $Z_A$ and $X_A X_B$ survive as the logical operators of $M$; $X_A$ and $X_B$ separately do not. Measuring the seam in the $X$ basis splits $M$ back into $A$ and $B$.

## 3 Circuit

The package `lattice_surgery_stim` builds the patch geometry, a memory circuit for any set of patches, and the surgery circuit: $d$ rounds on $A$ and $B$, $m$ rounds on $M$, $d$ rounds on $A$ and $B$, and a final $Z$ readout. Detectors compare each check with its previous round; the new checks have none in their first round, and a grown check that shrinks at the split is compared together with its two seam outcomes. Observable 0 is the parity of the new checks in the first merged round, and observables 1, 2 are $Z_A$, $Z_B$ at readout. The memory circuit matches Stim's `surface_code:rotated_memory_z` in logical error rate at $d = 3, 5$. Stim's shortest undetectable logical error has weight $\min(m, d)$ for $m = 1, \ldots, d$: with fewer than $d$ merged rounds, repeated measurement errors on one new check flip the parity unseen. This is why the merge lasts $d$ rounds.

## 4 Results




| $d$ | $p$ | surg. ZZ | surg. any | idle any |
|---|---|---|---|---|
| 3 | 0.002 | 1.12% | 1.92% | 1.72% |
| 3 | 0.004 | 4.13% | 7.14% | 6.49% |
| 3 | 0.006 | 8.39% | 14.57% | 13.12% |
| 5 | 0.002 | 0.34% | 0.62% | 0.65% |
| 5 | 0.004 | 2.46% | 4.41% | 4.77% |
| 5 | 0.006 | 7.59% | 13.58% | 13.70% |

| | surgery | | idle | |
|---|---|---|---|---|
| | $d$=3 | $d$=5 | $d$=3 | $d$=5 |
| qubits | 41 | 109 | 34 | 98 |
| rounds | 9 | 15 | 9 | 15 |
| detectors | 152 | 744 | 144 | 720 |

Failure rates over $10^5$ shots per point, with $m = d$; the baseline is the two patches idle for the same $3d$ rounds. "ZZ" means the parity was mispredicted, "any" that at least one observable was. At $p = 0.002$, $d = 5$ fails about a third as often as $d = 3$; at $p = 0.006$ the two are nearly equal, so the crossing is a little above that. The surgery fails about 10% more often than the baseline at $d = 3$ and is comparable to it at $d = 5$. A possible reason, not tested: during the merge an $X$ chain that flips $Z_A$ must cross the width $2d + 1$ instead of $d$. The surgery uses $2(2d+1)d-1$ qubits and costs $d$ rounds in time; gates are $Z$- and $X$-basis reset and measurement, H and CNOT.

## 5 Limitations

Only the smooth merge is implemented. Only $Z$-basis preparation is simulated, so the experiment tests the parity outcome and the survival of $Z_A$, $Z_B$, but not the coherence of $X_A X_B$, which needs an $X$-basis run. Noise is uniform depolarizing with one strength $p$. Only $d = 3, 5$ and three values of $p$ were run, so no threshold is claimed.

**Use of AI assistance.** The code in `src/` and the experiment script were written by the author, with specifications and review in words from an AI assistant (Claude). The tests `test_circuit.py` and `test_merge.py`, the plotting code, the figures and the slides were written by the assistant, and this summary was drafted by it and checked by the author. The full derivation is in `merge_note.pdf`.